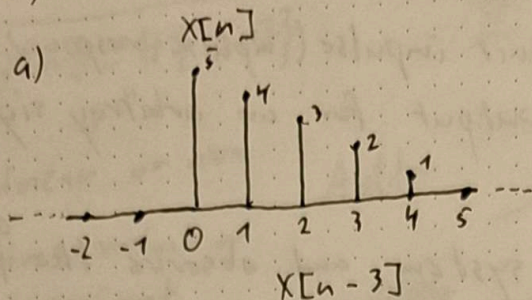


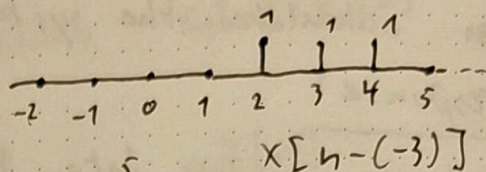
Problem 1)

Digsig Problem set 1

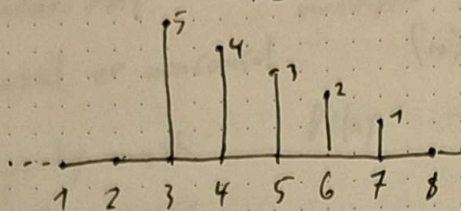
1) a)



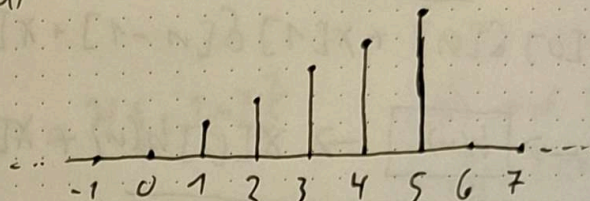
$y[n]$



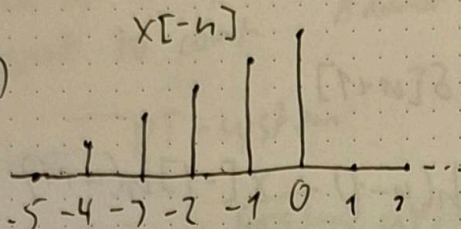
b)



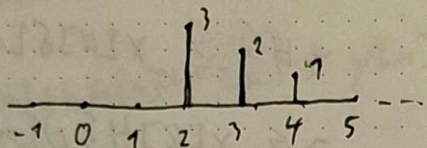
d) $x[5-n] = x[-(n-5)]$



c)



e)



f) $x[n] = (5-n) [\delta[n] + \delta[n-1] + \delta[n-2] + \delta[n-3] + \delta[n-4]]$

g) $y[n] = u[n-2] - u[n-5]$

h) $E_x = \sum_{n=-\infty}^{\infty} |x[n]|^2 = 5^2 + 4^2 + 3^2 + 2^2 + 1^2 = \underline{\underline{55}}$

Problem 2)

a)

Since the discrete time signal is periodic in the frequency domain with $f_1 \in [-\frac{1}{2}, \frac{1}{2})$ or $[0, 1)$ we get that:

$$f_1 = \frac{F_1}{F_s} \implies F_1 = f_1 \cdot F_s = [-3000, 3000) \text{ Hz}$$

b)

The code i used:

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
import sounddevice as sd

F_S = 1000
f_1 = 0.3
F_1 = f_1 * F_S

t_total = 4

N = np.arange(0, t_total, 1/F_S) # Getting F_S samples from 0 to 4 seconds

def x_func(t):
    return np.array(np.cos(2 * np.pi * F_1 * t))

x = x_func(N) # 4 seconds of x[n]

sd.default.device = 7
sd.play(x)
sd.wait()
```



c)

When i use a total of 4 seconds, i get a short light "beep" for $F_s = 1000$ Hz and a slightly longer one for $F_s = 3000$ Hz. With $F_s = 12000$ Hz I can hear it "flicker" a bit. See b) for code i used.

d)

For $F_1 = 1000$ Hz i got a really high pitch tone, for $F_1 = 6000$ Hz i got a slightly deeper tone. But with $F_1 = 3000$ Hz the tone got really dark. I think this has to do with the periodicity from the normalised frequency, as $f_1 = 0.75$ or 0.25 for $1 - f_1$ for $F_1 = 6000$ Hz, while for 1000 Hz $f_1 = 0.125$ and for 3000 Hz $f_1 = 0.375$ which is alot closer to the edge of 0.5 .

Problem 3)

(a) $y[n] = x[n] - x^2[n - 1]$

(b) $y[n] = nx[n] + 2x[n - 2]$

(c) $y[n] = x[n] - x[n - 1]$

(d) $y[n] = x[n] + 3x[n + 4]$

a)

Its time-invariant, as there is no factor n outside of the $x[n]$ that the system depends on. Its causal as it depends on current or past values. $x^2[n - 1]$ is non-linear as its squared.

b)

Its linear, as all factors in the system is linear so the superposition principle still holds. Its causal as it depends on future values in $2x[n - 2]$. Its time-variant because it has a factor n in $nx[n]$.

c)

Its linear, time-invariant and causal. All linear factors so superposition counts and no n factors outside of the $x[n]$. Its causal since it depends current or past values..

d)

This is a linear, time-invariant and non-causal system. All linear factors so superposition counts, no n factors outside of the $x[n]$ and it depends on future values in $3x[n + 4]$.

Problem 4)

4) a) $x[n] + 2x[n-1] + x[n-2] - 0,9y[n-1] + x[n] = y[n]$

$\Rightarrow h_1[n] = \delta[n] + 2\delta[n-1] + \delta[n-2]$

$\begin{cases} x[n] = \delta[n] \\ \Rightarrow y[n] = h[n] \end{cases}$

$\Rightarrow h_1[n] = \begin{cases} 1 & n=0 \\ 2 & n=1 \\ 1 & n=2 \\ 0 & \text{otherwise} \end{cases}$

$\Rightarrow h_2[n] = \delta[n] - 0,9h[n-1]$

For causal systems $h[n] = 0$ for $n < 0$

$\Rightarrow h_2[n] = \begin{cases} 1 - 0,9h[-1] = 1 & n=0 \\ -0,9h[0] = -0,9 & n=1 \\ -0,9h[1] = (-0,9)^2 & n=2 \\ -0,9h[2] = (-0,9)^3 & n=3 \\ \vdots \\ -0,9h[n-1] = (-0,9)^n & n \geq 0 \end{cases}$

$\Rightarrow h_2[n] = \underline{\underline{(-0,9)^n u[n]}}$

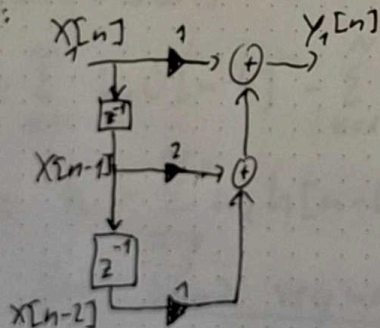
b) The first system is FIR as it has a finite $h[n]$ length of 3.
The second system is IIR as it has a $h[n]$ with infinite support.

c) $|y[n]| = M_x \sum_{k=-\infty}^{\infty} |h[k]| < \infty$ where $|x[n]| \leq M_x < \infty$

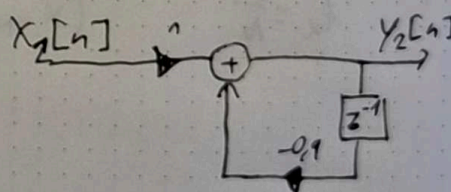
$|y_1[n]| = M_x (1+2+1) = M_x \cdot 4 \Rightarrow \text{stable}$

$|y_2[n]| = M_x \sum_{k=0}^{\infty} (-0,9)^k = M_x \frac{(-0,9)^0}{1 - (-0,9)^1} = M_x \frac{1}{1,9} \Rightarrow \text{stable}$

d) $y_1[n]$:

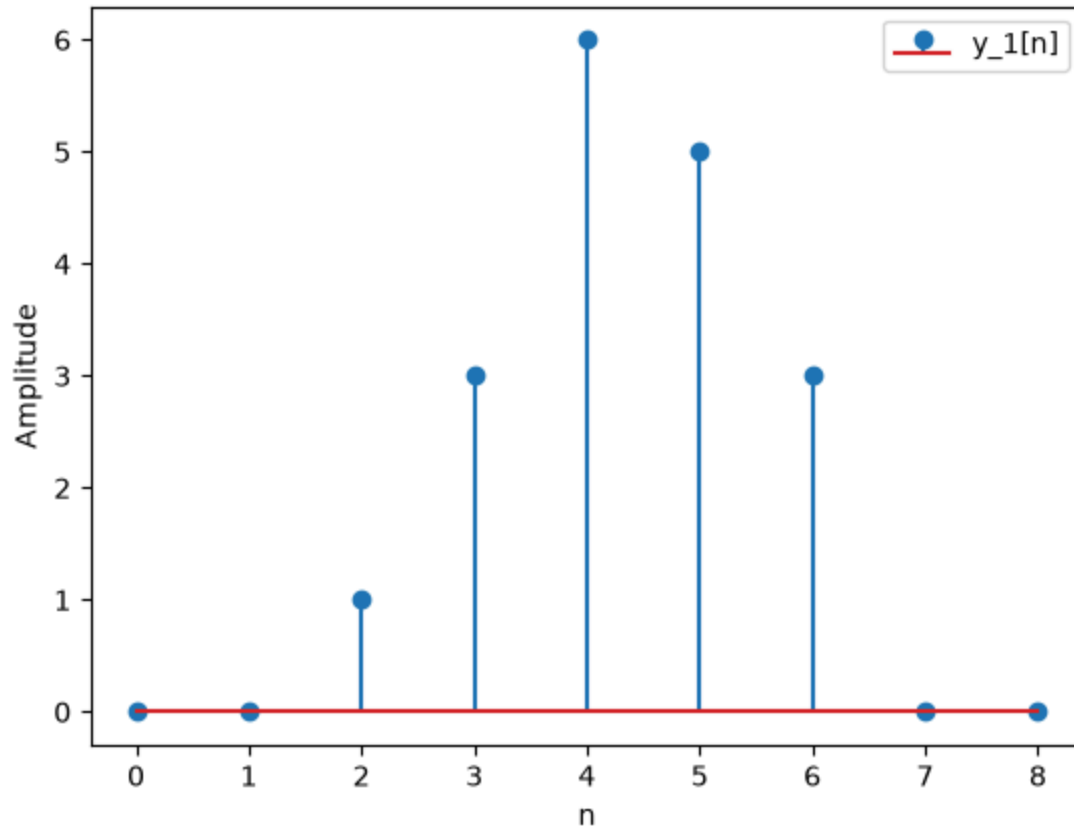


$y_2[n]$:



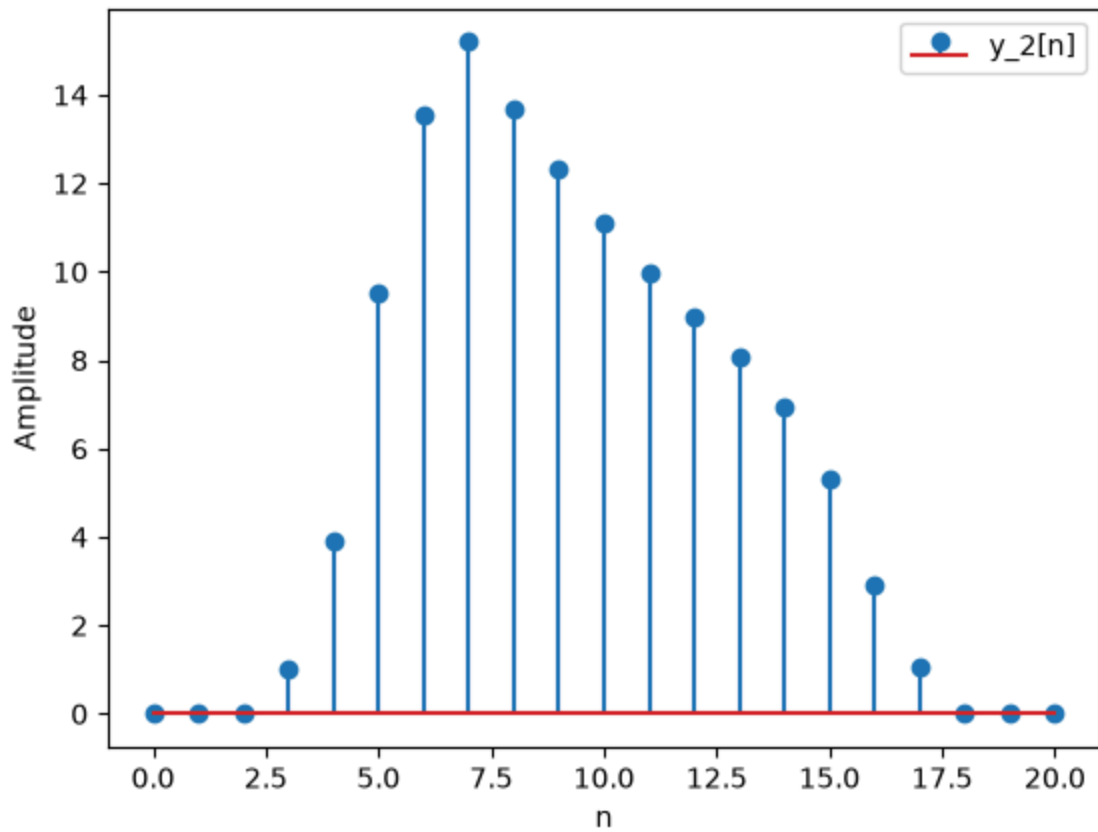
Problem 5)

a)



$$y_1[n] = [1, 3, 6, 5, 3]$$

b)



c)

The length of y_1 is given by:

$$\text{len}(y_1) = \text{len}(x) + \text{len}(h_1) - 1$$

And then y_2 is given by:

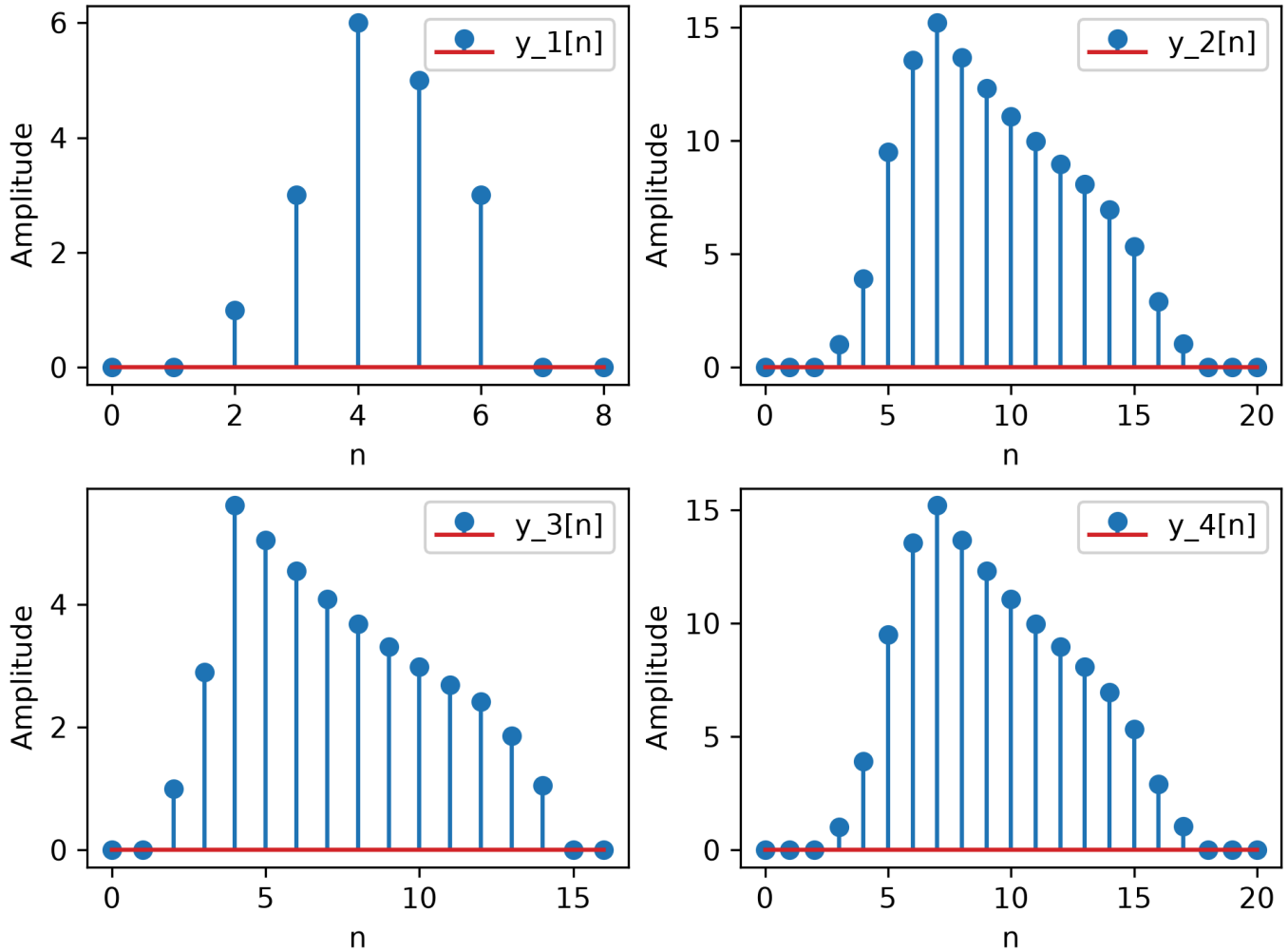
$$\text{len}(y_2) = \text{len}(y_1) + \text{len}(h_2) - 1$$

Which gives:

$$(3 + 3 - 1) + 11 - 1 = 15$$

d)

Swapping $h_1[n]$ and $h_2[n]$ and getting $y_3[n]$ and $y_4[n]$ instead:



We can clearly see that $y_1[n]$ and $y_3[n]$ are different as they are the convolution of $x[n]$ with $h_1[n]$ and $h_2[n]$ respectively. However we can see that associative property of convolution from $y_2[n] = y_4[n]$ as the order of the subsystems in a system only matters for the outputs in between the subsystems. The total output does not.

Code:

```

import numpy as np
import matplotlib.pyplot as plt

# a)
x = np.array([0, 1, 2, 3, 0,])

h_1 = np.array([0, 1, 1, 1, 0,])

h_2 = np.array([0] + [0.9**n for n in range(0, 11, 1)] + [0])

y_1 = np.convolve(x, h_1, "full")

n_1 = np.arange(0, len(y_1))

fig_1, ax1 = plt.subplots()
ax1.stem(n_1, y_1, label="y_1[n]")
ax1.set_ylabel("Amplitude")
ax1.set_xlabel("n")
ax1.legend()
plt.savefig("problem5_a.png")

# b)
y_2 = np.convolve(y_1, h_2, "full")

n_2 = np.arange(0, len(y_2))

fig_2, ax2 = plt.subplots()
ax2.stem(n_2, y_2, label="y_2[n]")
ax2.set_ylabel("Amplitude")
ax2.set_xlabel("n")
ax2.legend()
plt.savefig("problem5_b.png")

# d)
y_3 = np.convolve(x, h_2, "full")

n_3 = np.arange(0, len(y_3))

y_4 = np.convolve(y_3, h_1, "full")

n_4 = np.arange(0, len(y_4))

fig, ax = plt.subplots(2,2, layout="constrained")
ax[0,0].stem(n_1, y_1, label="y_1[n]")
ax[0,0].set_ylabel("Amplitude")
ax[0,0].set_xlabel("n")
ax[0,0].legend()

```



```
ax[0,1].stem(n_2, y_2, label="y_2[n]")
ax[0,1].set_ylabel("Amplitude")
ax[0,1].set_xlabel("n")
ax[0,1].legend()
```

```
ax[1,0].stem(n_3, y_3, label="y_3[n]")
ax[1,0].set_ylabel("Amplitude")
ax[1,0].set_xlabel("n")
ax[1,0].legend()
```

```
ax[1,1].stem(n_4, y_4, label="y_4[n]")
ax[1,1].set_ylabel("Amplitude")
ax[1,1].set_xlabel("n")
ax[1,1].legend()
```

```
plt.savefig("problem5_d.png", dpi=300)
```

